INF10025 Data Management and Analytics

Task 5– Credit Submission

Student Name: Truong Ngoc Gia Hieu
Student Number (VietNam/Australia): SWS01217/ 105565520

A. Task Credit5a

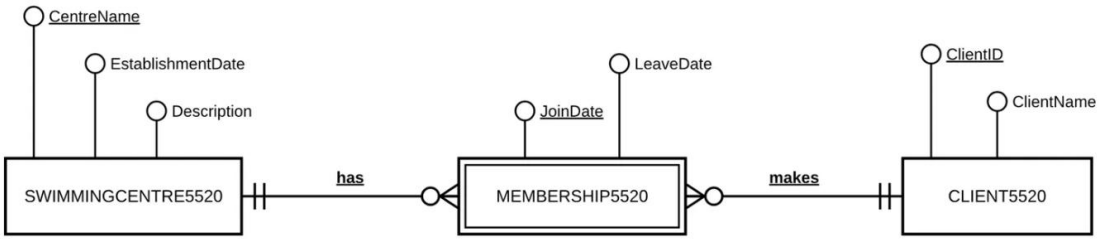


Figure 1: Entity Relationship Diagram (ERD) for Task Credit 5a.

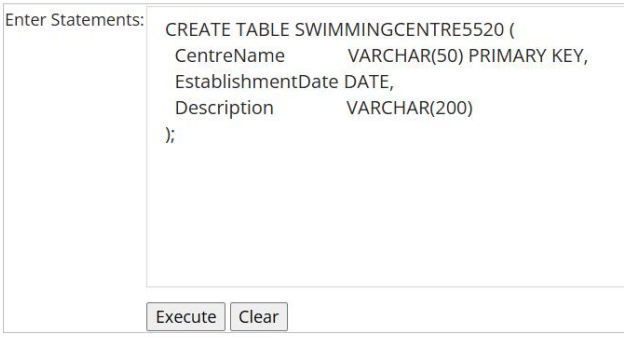


Figure 2: SQL statement for creating the SWIMMINGCENTRE5520 table.

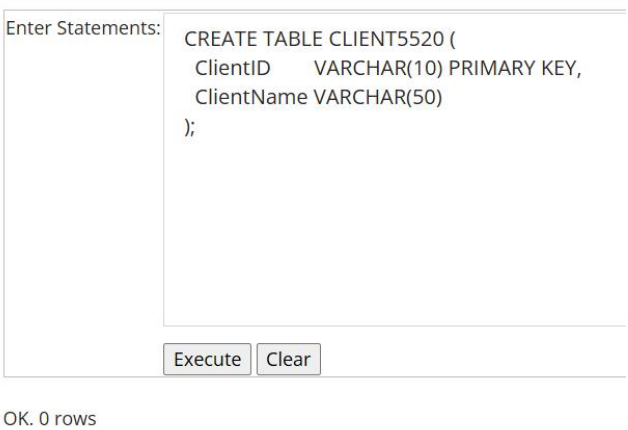


Figure 3: SQL statement for creating the CLIENT5520 table.

Enter Statements:

```
CREATE TABLE MEMBERSHIP5520 (
  CentreName VARCHAR(50),
  ClientID   VARCHAR(10),
  JoinDate   DATE,
  LeaveDate  DATE,
  PRIMARY KEY (CentreName, ClientID, JoinDate),
  FOREIGN KEY (CentreName) REFERENCES
SWIMMINGCENTRE5520(CentreName),
  FOREIGN KEY (ClientID)   REFERENCES CLIENT5520(ClientID)
);
```

Execute Clear

OK. 0 rows

Figure 4: SQL statement for creating the MEMBERSHIP5520 table.




```
SELECT TABLE_NAME
FROM ALL_TABLES
WHERE OWNER =
(SELECT USER FROM
DUAL)
```

TABLE_NAME
ACTOR5520
ASSIGNMENT5520
ATTENDEE5520
CAMPAIGN5520
CAR5520
CASTING5520
CLIENT5520
COLOURTYPE5520
COMEDIAN5520
CUSTOMER5520
DRIVER5520
JOB5520
MEMBERSHIP5520
MOVIE5520
PLAYER5520
RATING5520
RATINGSHOW5520
SHOW5520
SWIMMINGCENTRE5520

OK. 19 rows selected.

Figure 5: SQL statement for listing all created tables in the database.

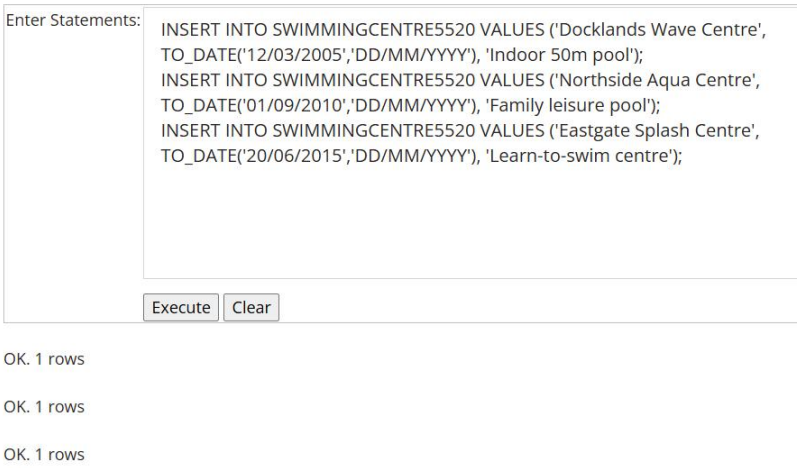


Figure 6: SQL statements for inserting records into the SWIMMINGCENTRE5520 table.



SELECT * FROM SWIMMINGCENTRE5520

CENTRENAME	ESTABLISHMENTDATE	DESCRIPTION
Docklands Wave Centre	12-MAR-05	Indoor 50m pool
Northside Aqua Centre	01-SEP-10	Family leisure pool
Eastgate Splash Centre	20-JUN-15	Learn-to-swim centre

OK. 3 rows selected.

Figure 7: SQL query displaying all records in the SWIMMINGCENTRE5520 table.

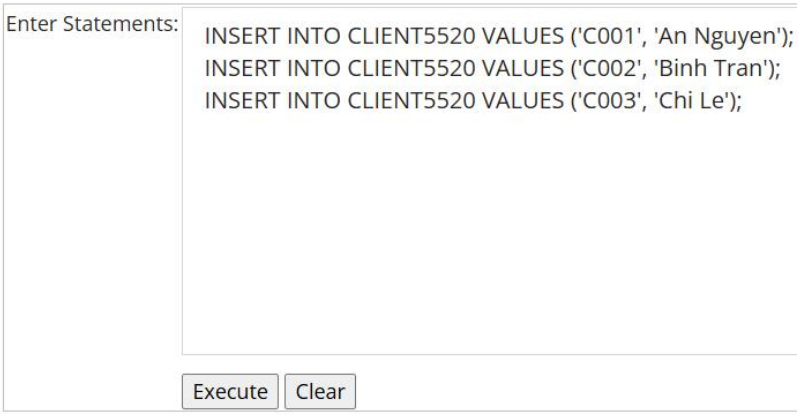


Figure 8: SQL statements for inserting records into the CLIENT5520 table.

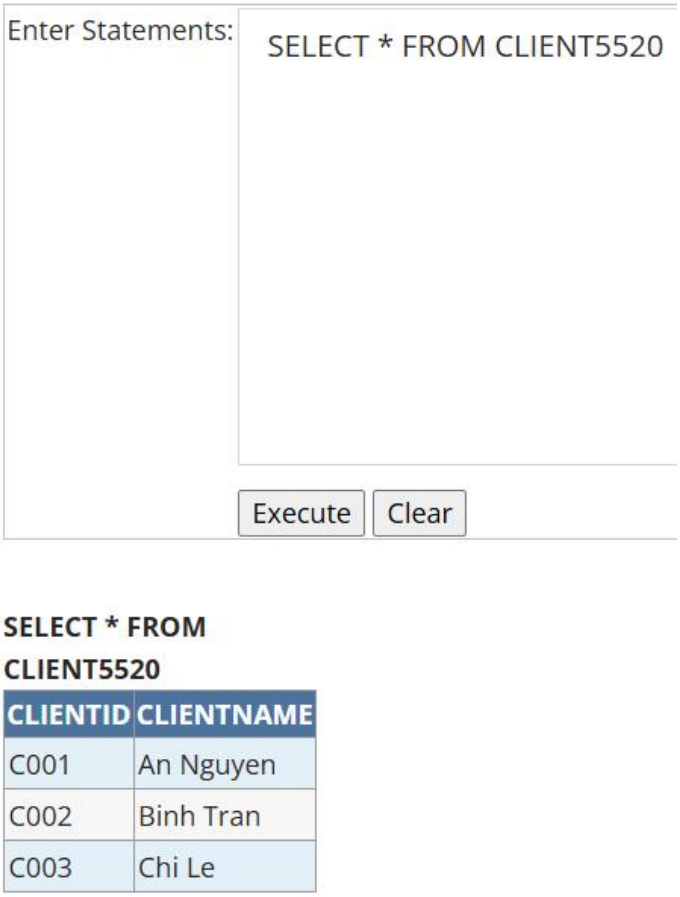


Figure 9: SQL query displaying all records in the CLIENT5520 table.

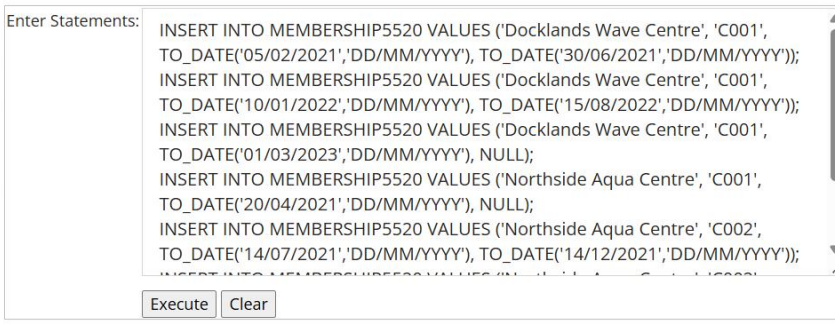
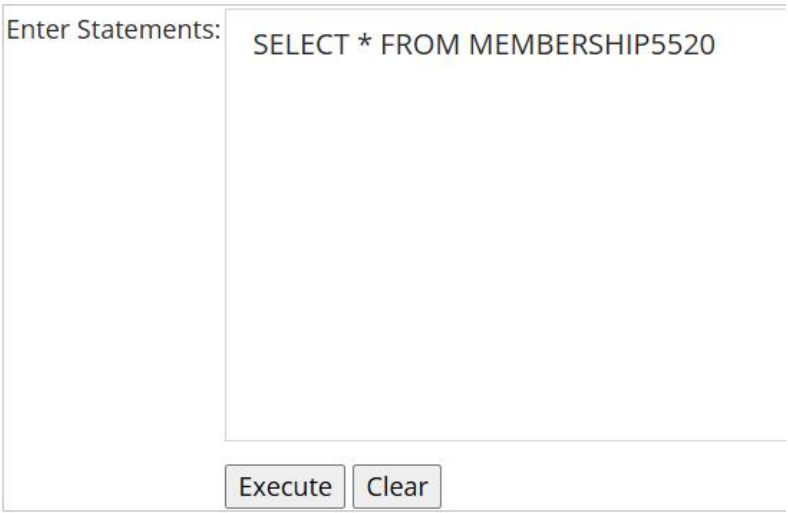


Figure 10: SQL statements for inserting records into the MEMBERSHIP5520 table.



SELECT * FROM MEMBERSHIP5520

CENTRENAME	CLIENTID	JOINDATE	LEAVEDATE
Docklands Wave Centre	C001	05-FEB-21	30-JUN-21
Docklands Wave Centre	C001	10-JAN-22	15-AUG-22
Docklands Wave Centre	C001	01-MAR-23	
Northside Aqua Centre	C001	20-APR-21	
Northside Aqua Centre	C002	14-JUL-21	14-DEC-21
Northside Aqua Centre	C002	02-MAY-22	
Eastgate Splash Centre	C003	09-SEP-22	09-MAR-23

OK. 7 rows selected.

Figure 11: SQL query displaying all records in the MEMBERSHIP5520 table.

B. Task Credit5b

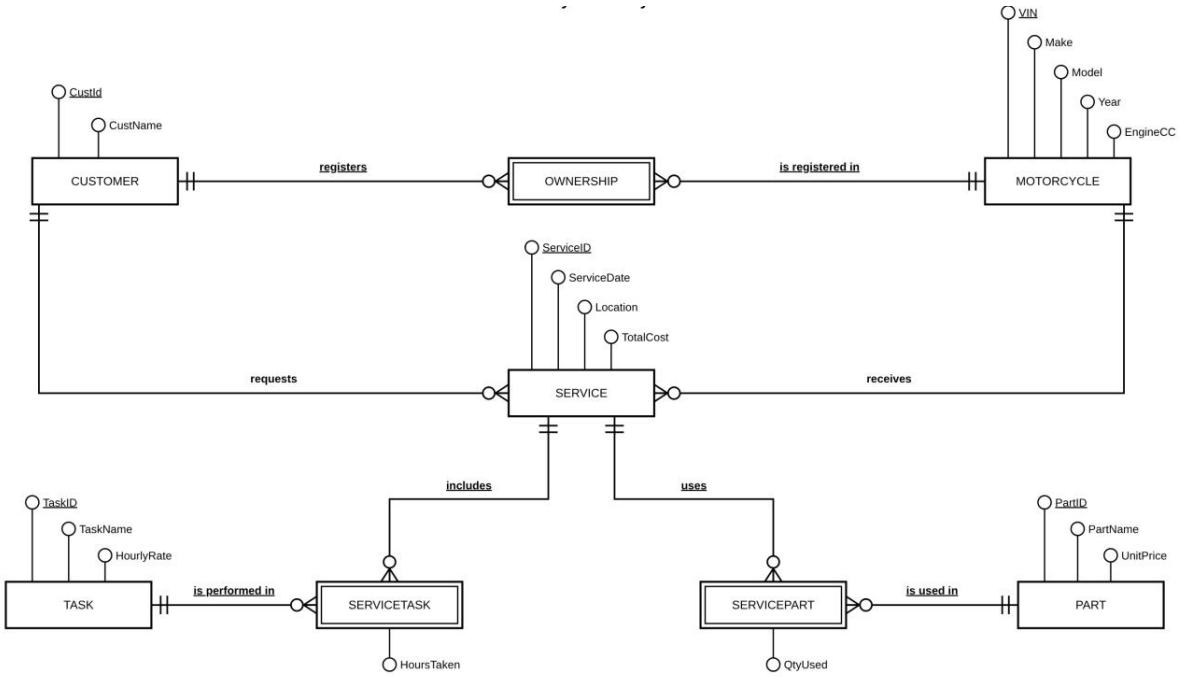


Figure 12: Entity Relationship Diagram (ERD) for Tiny's Motorcycle Maintenance.

C. Task Credit5c

CUSTOMER5520 (CustId, CustName)
Primary Key (CustId)

MOTORCYCLE5520 (VIN, Make, Model, Year, EngineCC)
Primary Key (VIN)

OWNERSHIP5520 (CustId, VIN)
Primary Key (CustId, VIN)
Foreign Key (CustId) references CUSTOMER5520
Foreign Key (VIN) references MOTORCYCLE5520

SERVICE5520 (ServiceID, ServiceDate, Location, TotalCost, CustId, VIN)
Primary Key (ServiceID)
Foreign Key (CustId) references CUSTOMER5520
Foreign Key (VIN) references MOTORCYCLE5520

TASK5520 (TaskID, TaskName, HourlyRate)
Primary Key (TaskID)

PART5520 (PartID, PartName, UnitPrice)
Primary Key (PartID)

SERVICETASK5520 (ServiceID, TaskID, HoursTaken)
Primary Key (ServiceID, TaskID)
Foreign Key (ServiceID) references SERVICE5520
Foreign Key (TaskID) references TASK5520

SERVICEPART5520 (ServiceID, PartID, QtyUsed)
Primary Key (ServiceID, PartID)
Foreign Key (ServiceID) references SERVICE5520
Foreign Key (PartID) references PART5520

Figure 13: Relational Schema for Tiny's Motorcycle Maintenance.

D. Task Credit5d

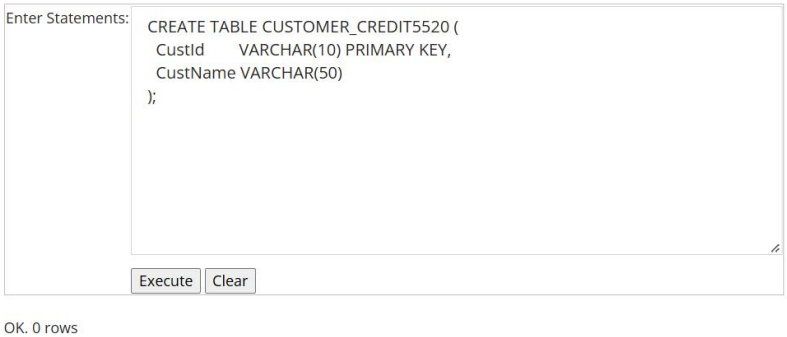
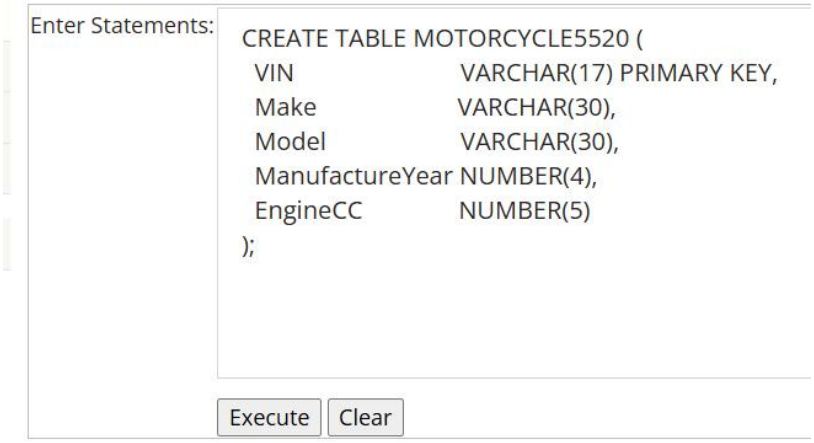


Figure 14: SQL statement for creating the CUSTOMER_CREDIT5520 table.



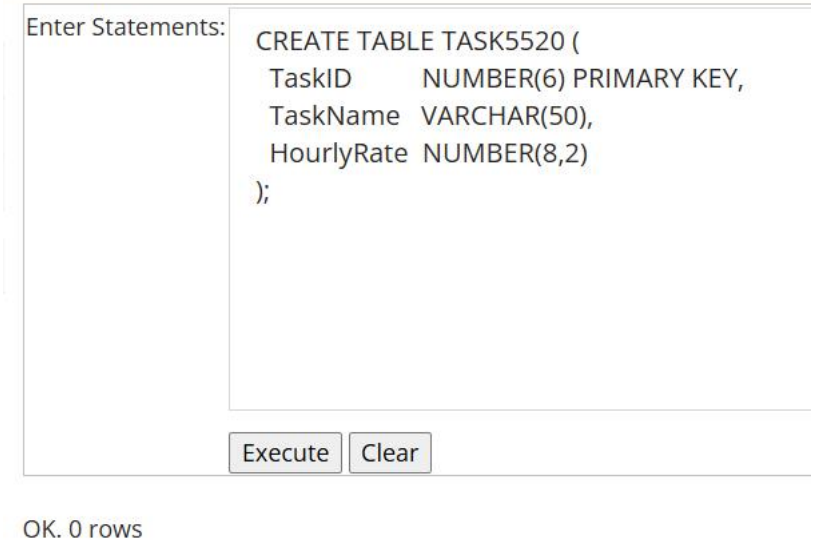


Figure 16: SQL statement for creating the TASK5520 table.

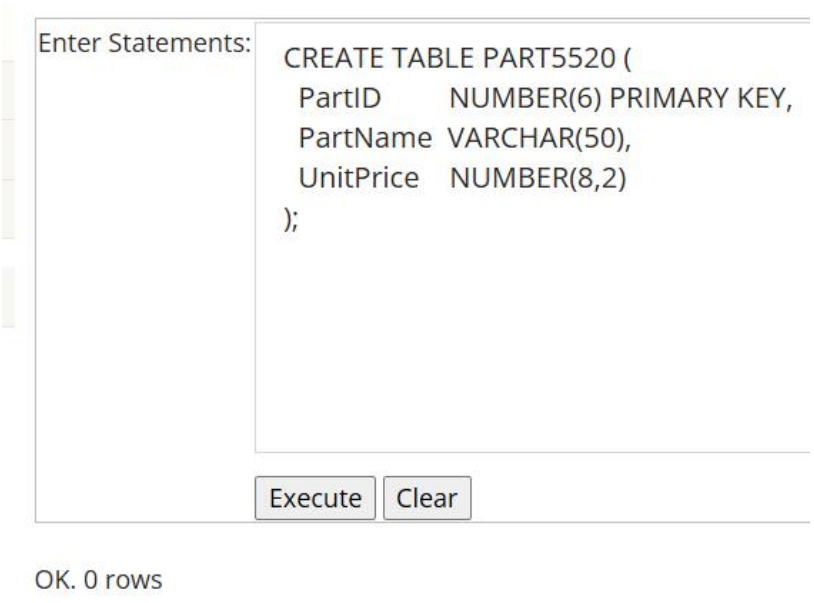


Figure 17: SQL statement for creating the PART5520 table.

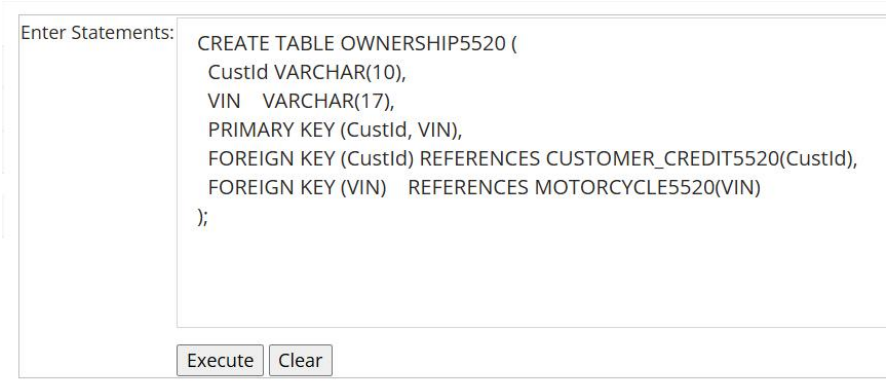


Figure 18: SQL statement for creating the OWNERSHIP5520 table.

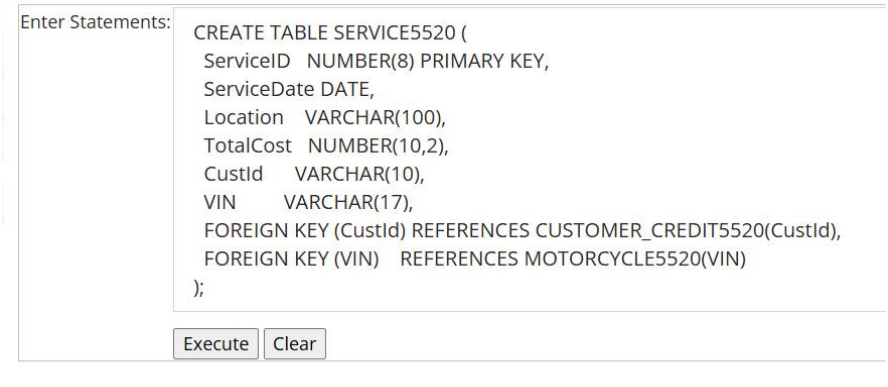


Figure 19: SQL statement for creating the SERVICE5520 table.



Figure 20: SQL statement for creating the SERVICETASK5520 table.



Figure 21: SQL statement for creating the SERVICEPART5520 table.



Figure 22: SQL statement for listing all created tables in the database after creating the Credit 5d tables.

E. Task Credit5e

The 1NF table from Pass 5e has the composite primary key (CustId, CarRego, StartDate). To reach 3NF, the partial dependencies were removed first: Name and Phone depend only on CustId, and MakeModel depends only on CarRego. After that, no transitive dependency remains, so every non-key attribute depends on the key, the whole key and nothing but the key.

CUSTOMER (CustId, Name, Phone)

CustId	Name	Phone
125	John Coles	0401112233
278	Erin Trump	0466121455
721	Lynda Rodgers	0423544117

Table 1: CUSTOMER relation in 3NF (primary key: CustId).

CAR (CarRego, MakeModel)

CarRego	MakeModel
1AU8HK	Mazda 3
1LM3AB	Hyundai i30
1KA2CA	Toyota Camry
1CZ8JK	Mazda 3

Table 2: CAR relation in 3NF (primary key: CarRego).

RENTAL (CustId, CarRego, StartDate, ReturnDate)

CustId	CarRego	StartDate	ReturnDate
125	1AU8HK	31/08/2020	7/09/2020
125	1LM3AB	14/11/2020	21/11/2020
278	1AU8HK	12/09/2020	19/09/2020

278	1KA2CA	1/10/2020	8/10/2020
278	1CZ8JK	10/11/2020	12/11/2020
278	1AU8HK	26/11/2020	1/12/2020
721	1LM3AB	10/09/2020	13/09/2020

Table 3: RENTAL relation in 3NF (primary key: CustId, CarRego, StartDate; CustId references CUSTOMER and CarRego references CAR).

F. Task Credit5f

SQL statements to create the tables and insert the initial data

```
CREATE TABLE PRODUCT5520 (
  ProdID      VARCHAR(10) PRIMARY KEY,
  Brand       VARCHAR(30),
  Description VARCHAR(100),
  PricePerOne NUMBER(8,2),
  QtyInStock  NUMBER(5) CHECK (QtyInStock >= 0)
);

CREATE TABLE ACTION5520 (
  ActionID       NUMBER(6) PRIMARY KEY,
  ActionDateTime DATE,
  ActionType     VARCHAR(20),
  ProdID         VARCHAR(10),
  ProdQty        NUMBER(5),
  ProdCost       NUMBER(10,2),
  FOREIGN KEY (ProdID) REFERENCES PRODUCT5520(ProdID)
);

INSERT INTO PRODUCT5520 VALUES ('G43546', 'Gucci', 'Leather mid-heel pump',
1050.00, 12);
COMMIT;
```

Transaction 1 - Purchase (ActionID 1008)

```
INSERT INTO ACTION5520 VALUES (1008, TO_DATE('21/01/2021','DD/MM/YYYY'),
'Purchase', 'G43546', 2, 2100.00);
UPDATE PRODUCT5520 SET QtyInStock = QtyInStock - 2 WHERE ProdID = 'G43546';
COMMIT;
```

Transaction 2 - Return (ActionID 1026)

```
INSERT INTO ACTION5520 VALUES (1026, TO_DATE('23/01/2021','DD/MM/YYYY'),
'Return', 'G43546', -1, 1050.00);
UPDATE PRODUCT5520 SET QtyInStock = QtyInStock - (-1) WHERE ProdID =
'G43546';
COMMIT;
```

Checking the result (QtyInStock should be 12 - 2 + 1 = 11)

```
SELECT * FROM PRODUCT5520;
SELECT * FROM ACTION5520;
```

A failed transaction and ROLLBACK (purchase of 50 pairs when only 11 are in stock)

```
INSERT INTO ACTION5520 VALUES (1030, TO_DATE('25/01/2021','DD/MM/YYYY'),
'Purchase', 'G43546', 50, 52500.00);
UPDATE PRODUCT5520 SET QtyInStock = QtyInStock - 50 WHERE ProdID =
'G43546';  -- fails: ORA-02290 (check constraint)
ROLLBACK;
```

Explanation

From my perspective, a sale or a refund goes smoothly when every step of the transaction succeeds. For a purchase, the system records a new row in the ACTION table and then reduces QtyInStock in the PRODUCT table. Similarly, for a return, it records a negative quantity and increases the stock. As a result, the two tables stay consistent, and the transaction is "committed", which means that the changes are saved permanently and become visible to other users.

However, an operation can go wrong if one step fails while the other succeeds, for example when the customer buys more pairs than the stock level allows, or when the system crashes after the ACTION row is inserted but before the stock is updated. Without protection, this would leave the transaction history and the stock level out of agreement. In this case, the transaction is "rolled back", which means that all changes made since the transaction started are undone and the database returns to its previous consistent state. Therefore, a transaction works as an all-or-nothing unit, which protects the accuracy of the data.